

DocQuery RAG-Based Document Q&A / Study Assistant "Stop searching your files. Just ask them."

1. Overview

DocQuery is a Retrieval-Augmented Generation (RAG) system that lets users upload PDFs and notes, then ask natural-language questions and receive answers strictly grounded in the uploaded material, with citations to the exact page. It is built to eliminate hallucination in private-document Q&A by never letting the model answer from general knowledge alone.

Key properties:

Answers are grounded — every claim traces back to a retrieved chunk. Every citation points to a specific document and page number. The system explicitly refuses to answer when no supporting evidence is found. Each user's document set is isolated — no cross-user data access.

2. Problem Statement

Students and professionals accumulate large volumes of unstructured PDFs, notes, textbooks, and research papers with no fast way to query across them. Finding a single fact often means manually scrolling through dozens of pages across multiple files.

General-purpose chatbots cannot solve this problem because:

They have no access to a user's private documents. They rely on parametric memory, which risks hallucinating answers that sound confident but are factually wrong. They cannot tell a user exactly where in a source document an answer came from.

DocQuery closes this gap between what the documents actually say, what the user needs to know, and whether the answer can be trusted.

3. Target Audience

Primary — Students Exam preparation across textbooks, lecture notes, and slides. Revising large syllabi spread across many PDFs. Getting quick, page-accurate answers instead of re-reading full chapters.

Secondary — Early-Career Professionals & Researchers Querying internal documentation, manuals, or reports. Cross-referencing multiple research papers. Working with private document sets that shouldn't be exposed to a general-purpose AI tool.

Future Enterprise Audience Corporate knowledge bases, legal/compliance document review, onboarding material, and customer-support knowledge assistants.

4. Prerequisites

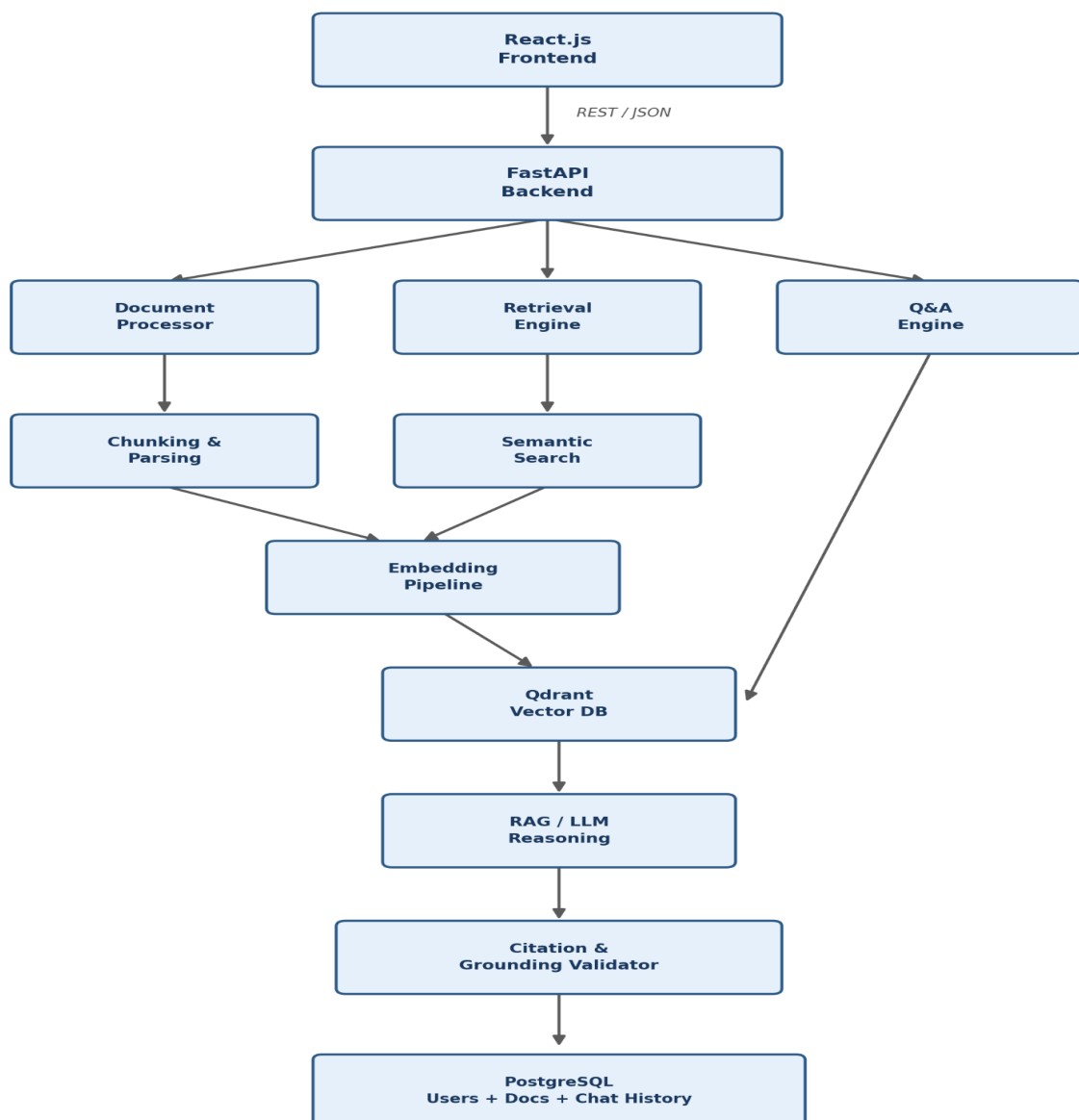
To run or develop DocQuery locally, the following are required:

Runtime Python 3.11+ Node.js 18+ (frontend build) Docker & Docker Compose

Accounts / Keys Gemini API key or OpenAI API key (answer generation) Embedding provider key (if using a hosted embedding API rather than a local Sentence Transformers model) Google Cloud Platform project (for deployment to Cloud Run — optional for local dev)

Infrastructure (local dev, via Docker Compose) PostgreSQL 15+ Qdrant (latest stable image)

5. System Architecture



6. Core Workflow

Stage 1 — Document Ingestion User uploads a PDF/DOCX. The system extracts text page-by-page, preserving page boundaries, headings, and (where present) tables, plus metadata: title, page count, upload timestamp.

Stage 2 — Chunking & Embedding Extracted text is split into semantically coherent chunks, each tagged with its source document ID and page number, then embedded into vector form and indexed.

Stage 3 — Retrieval A user question is embedded and matched against the vector store via semantic (and optionally keyword) search. Only the top-K most relevant chunks are passed forward — never the full document set.

Stage 4 — Grounded Answer Generation The LLM is prompted to answer using only the retrieved chunks and to cite the page(s) used. If retrieval confidence is below threshold, DocQuery returns "This isn't covered in the uploaded documents" instead of guessing.

7. Chunking Strategy & Parameters

Method: Recursive semantic chunking (paragraph/heading-aware, not fixed character cuts) using LangChain's RecursiveCharacterTextSplitter with a semantic-boundary preference.

Parameter | Value | Rationale
Chunk size | 500–800 tokens | Balances context completeness against retrieval precision
Chunk overlap | 100–150 tokens (≈20%) | Prevents loss of context at chunk boundaries
Splitting priority | Section > Paragraph > Sentence > Token | Avoids breaking ideas mid-thought
Minimum chunk size | 50 tokens | Discards near-empty fragments (headers, page numbers)
Metadata per chunk | document_id, page_number, section_title, chunk_index | Enables exact-page citation

8. Embedding & Vector Store Configuration

Embedding model: sentence-transformers/all-MiniLM-L6-v2 (local, default) or text-embedding-3-small (hosted, optional upgrade path)
Vector dimensions: 384 (MiniLM) / 1536 (OpenAI small embedding, if configured)
Similarity metric: Cosine similarity
Index type: HNSW (Qdrant default)
Top-K retrieved per query: 5–8 chunks (configurable)
Re-ranking: Optional cross-encoder re-ranking pass on top-20 candidates, reduced to top-5 before generation
Retrieval confidence threshold: Minimum cosine similarity of 0.55 for a chunk to be considered valid evidence; below this, the query is flagged as "insufficient evidence"

9. Performance Targets

Operation | Target Latency
Document upload → fully indexed (per 20-page PDF) | < 15 seconds
Query embedding | < 100 ms
Vector similarity search (top-K) | < 300 ms
End-to-end answer (retrieval + generation) | < 3 seconds
Concurrent documents per user (tested) | Up to 50 documents / ~5,000 pages

These are design targets for the current architecture, to be validated against real load testing before being cited as measured benchmarks.

10. API Endpoints (REST)

Method | Endpoint | Description
POST | /api/auth/register | Create a new user account
POST | /api/auth/login | Authenticate and receive a JWT
POST | /api/documents/upload | Upload a PDF/DOCX for processing
GET | /api/documents | List all documents for the authenticated user
GET | /api/documents/{document_id} | Get metadata for a single document
DELETE | /api/documents/{document_id} | Delete a document and its indexed chunks
POST | /api/chat/sessions | Start a new Q&A session
POST | /api/chat/sessions/{session_id}/query | Submit a question and receive a grounded, cited answer
GET | /api/chat/sessions/{session_id}/history | Retrieve past Q&A turns for a session
GET | /api/chat/sessions/{session_id}/citations/{message_id} | Get the exact source chunks/pages behind an answer

All endpoints (except register/login) require a valid JWT in the Authorization header.

11. Database Schema

PostgreSQL — structured application data

users id (PK), email, password_hash, created_at

documents id (PK), user_id (FK → users), title, filename, page_count, upload_status, uploaded_at

chunks_metadata id (PK), document_id (FK → documents), page_number, section_title, chunk_index, vector_id (reference into Qdrant)

chat_sessions id (PK), user_id (FK → users), title, created_at

messages id (PK), session_id (FK → chat_sessions), role (user/assistant), content, created_at

citations id (PK), message_id (FK → messages), document_id (FK → documents), page_number, chunk_id (FK → chunks_metadata), similarity_score

Qdrant — vector store

Collection per deployment (or per user, depending on isolation strategy), storing: vector embedding, document_id, page_number, chunk_index as payload fields for filtered retrieval.

12. Tech Stack

Frontend: React.js, TypeScript, Tailwind CSS, PDF.js (in-app page preview / citation jump-to-page)
Backend: Python, FastAPI
AI / NLP: LangChain, Sentence Transformers / OpenAI Embeddings, Hugging Face Transformers, Gemini API / OpenAI API, Pydantic
Retrieval: Qdrant, hybrid (semantic + keyword) search, cross-encoder re-ranking
Document

Processing: PyMuPDF (PDF), python-docx (DOCX), Tesseract OCR (scanned documents, optional) Database: PostgreSQL (structured data), Qdrant (vector search) Auth & Security: JWT authentication, Argon2 password hashing, HTTPS, per-user document isolation, encryption at rest for stored documents DevOps / Cloud: Docker, Docker Compose, Google Cloud Run, Google Cloud Platform

13. Setup (Local Development)

14. Clone the repository and copy `.env.example` to `.env`; fill in database URL, Qdrant URL, and LLM/embedding API keys.

15. Run `docker compose up -d` to start PostgreSQL and Qdrant.

16. Backend: create a virtual environment, `pip install -r requirements.txt`, run migrations, then `uvicorn app.main:app --reload`.

17. Frontend: `npm install`, then `npm run dev`.

18. Access the app at the local frontend URL; the backend API is served separately and documented via FastAPI's auto-generated OpenAPI docs (`/docs`).

19. Example End-to-End Scenario

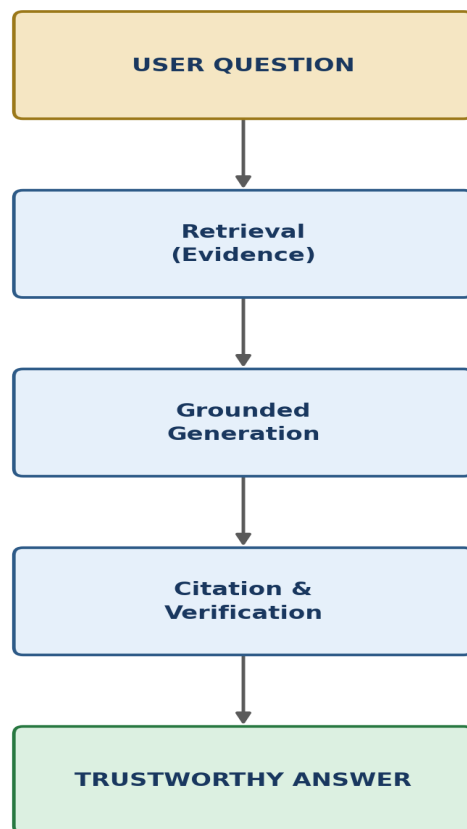
A student uploads a 120-page Machine Learning textbook and asks:

"Explain the difference between bagging and boosting, and tell me which pages cover it."

Retrieved evidence: Chapter 6, Page 142 (bagging definition), Page 144 (bootstrap sampling), Page 149 (boosting definition), Page 152 (AdaBoost example).

DocQuery answer: "Bagging trains multiple models in parallel on bootstrapped samples of the data and averages their predictions to reduce variance (p. 142, p. 144). Boosting, in contrast, trains models sequentially, where each new model focuses on correcting the errors of the previous one (p. 149), as illustrated through the AdaBoost algorithm (p. 152)."

15. Three-Layer Grounding Model



An answer is not considered valid merely because the model generated fluent text — it must progress through Question → Evidence → Grounded Answer → Verified Citation. If any stage fails to find sufficient evidence, DocQuery surfaces that explicitly instead of guessing.

16. What Makes DocQuery Unique

General AI Chatbot | DocQuery Answers from general training | Answers from your own documents May hallucinate confidently | Refuses when evidence is missing No source traceability | Exact page-level citations No document isolation | Per-user private document store One-shot generic answers | Retrieval -grounded, verifiable answers Good for general knowledge | Built for exam prep & private material